

Software Architecture and Design Specification

Project: Railway Reservation System

Team No: Team 1 (Jackfruit Problem), Section 5C

Version: 1.0

Authors: Farheen Akhtar (PES2UG24CS164), HANUMANTHA L (PES2UG24CS179), GIRISH JAYAN SENTHILKUMAR (PES2UG24CS170), MEENAKSHI P (PES2UG25CS811)

Date: 15-09-2026

Status: Draft

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Team 1: Farheen Akhtar Hanumantha L Girish Jayan Senthilkumar Meenakshi P	Initial draft

Approvals

Role	Name	Signature/Date
Project Guide	Dr. Pradeep Kumar Kenny S	14-09-2026

1. Introduction

1.1 Purpose

This document specifies the software architecture and design of the Railway Reservation System (RRS) defined in SRS v1.0 (Team 1, 05-09-2026). It translates the 22 functional requirements (RRS-F-001 to RRS-F-022), 5 non-functional requirements (RRS-NF-001 to RRS-NF-005), and 5 security requirements (RRS-SR-001 to RRS-SR-005) of that SRS into an implementable C/C++ design.

1.2 Scope

Covers the passenger-facing operations defined in SRS v1.0 (registration and login, train search, seat booking, fare payment, ticket generation, cancellation and refund tracking, and booking history) and the administrative operations (train schedule and seat-inventory management, fare/quota configuration, reporting, and audit logging). Excludes actual payment-gateway settlement and the physical railway signalling/operations systems, which are treated as external interfaces, consistent with SRS Section 1.2.

1.3 Audience

Developers, Testers/QA, Course Instructor/Evaluators, and end-users (passengers and booking-counter staff), consistent with SRS Section 1.3.

1.4 Definitions

RRS (Railway Reservation System), PNR (Passenger Name Record), UI (User Interface), DB (Database/File Store), CLI (Command Line Interface), FR (Functional Requirement), NFR (Non-Functional Requirement), ADR (Architecture Decision Record).

2. Document Overview

2.1 How to use this document

Provides architectural deliverables including UML diagrams, an Architecture Decision Record (ADR) for the chosen pattern, technology stack, API/interface design, and a security threat model.

2.2 Related Documents

SRS v1.0 — Railway Reservation System, Team 1, 05-09-2026 (source of all RRS-F / RRS-NF / RRS-SR requirement IDs and of the use-case diagrams in Section 7). STP v1.0 — Software Test Plan. RTM — Requirements Traceability Matrix (SRS Section 8).

3. Architecture

3.1 Goals & Constraints

Goals: Seat search within 2 seconds (RRS-NF-001); seat consistency such that booked seats never exceed available seats (RRS-NF-002); durable persistence of booking and cancellation records across restarts (RRS-NF-003); a menu-driven interface usable without documentation (RRS-NF-004); portability across Windows and Linux (RRS-NF-005).

Constraints: Must be implemented in C/C++; limited to single-machine local file storage with no distributed database; seat inventory must remain consistent under concurrent-like access simulation; academic-project time and resource constraints (SRS Section 2.5).

3.2 Stakeholders & Concerns

- Passenger: general public with basic computer literacy; expects a simple booking flow and quick confirmation
- Booking Clerk / Admin: railway staff managing schedules, seat inventory, and manual bookings/cancellations
- System Administrator: manages user accounts, data backups, and system configuration
- Developers / QA: modularity and testability of each module against the SRS acceptance criteria

3.3 Component (UML) Diagram

Two structural views are given. The class diagram (Figure 1) shows the persistent data model — the entity classes, their attributes and operations, and the relationships and multiplicities between them. The component diagram (Figure 2) shows the runtime modules and their dependencies, named to match the Design Spec Module column of the RTM in SRS Section 8.

Figure 1 — UML Class Diagram

Railway Reservation System — UML Class Diagram

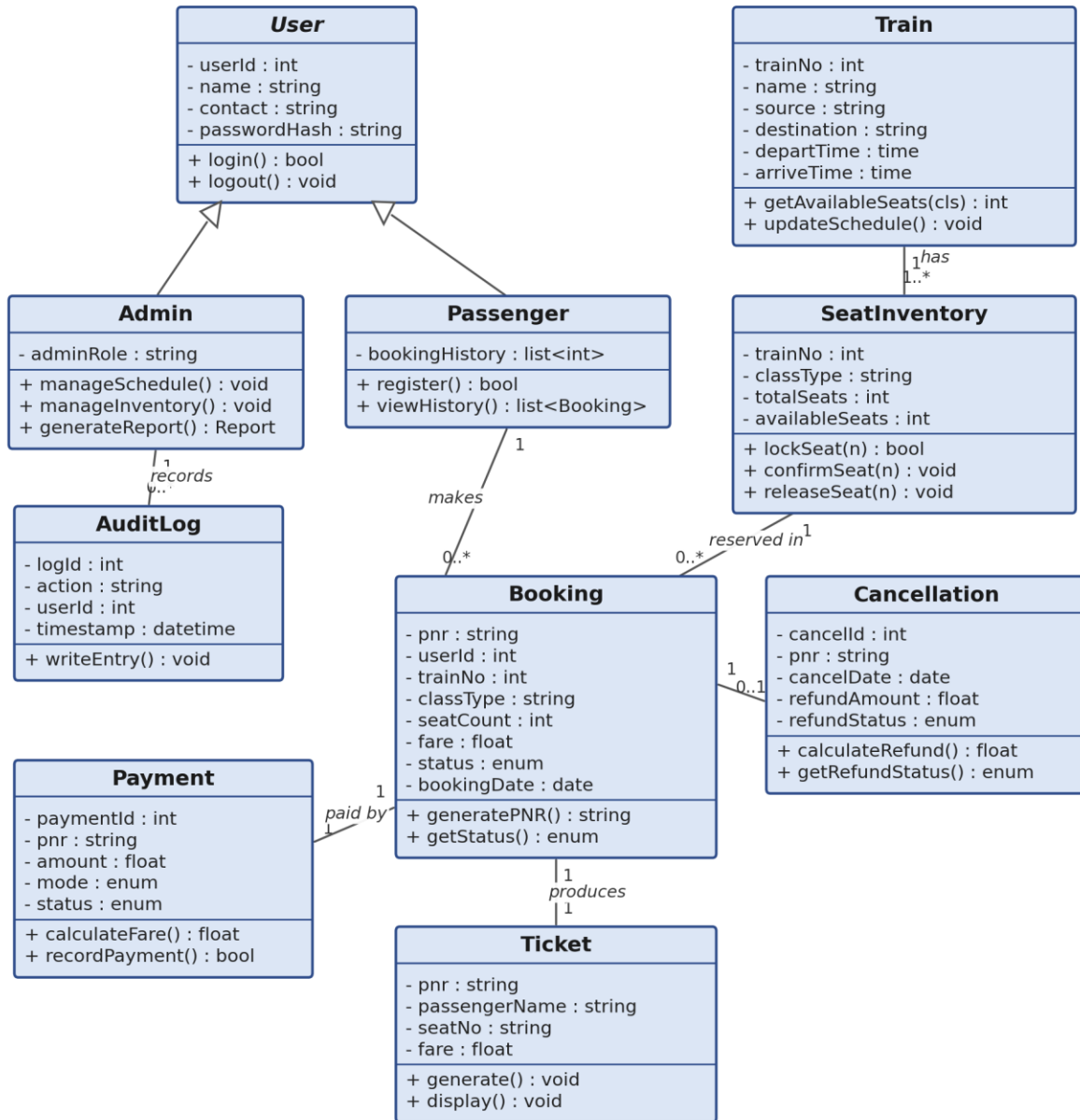


Figure 1: UML class diagram — entity classes, attributes, operations, and relationships

Figure 2 — Component Diagram

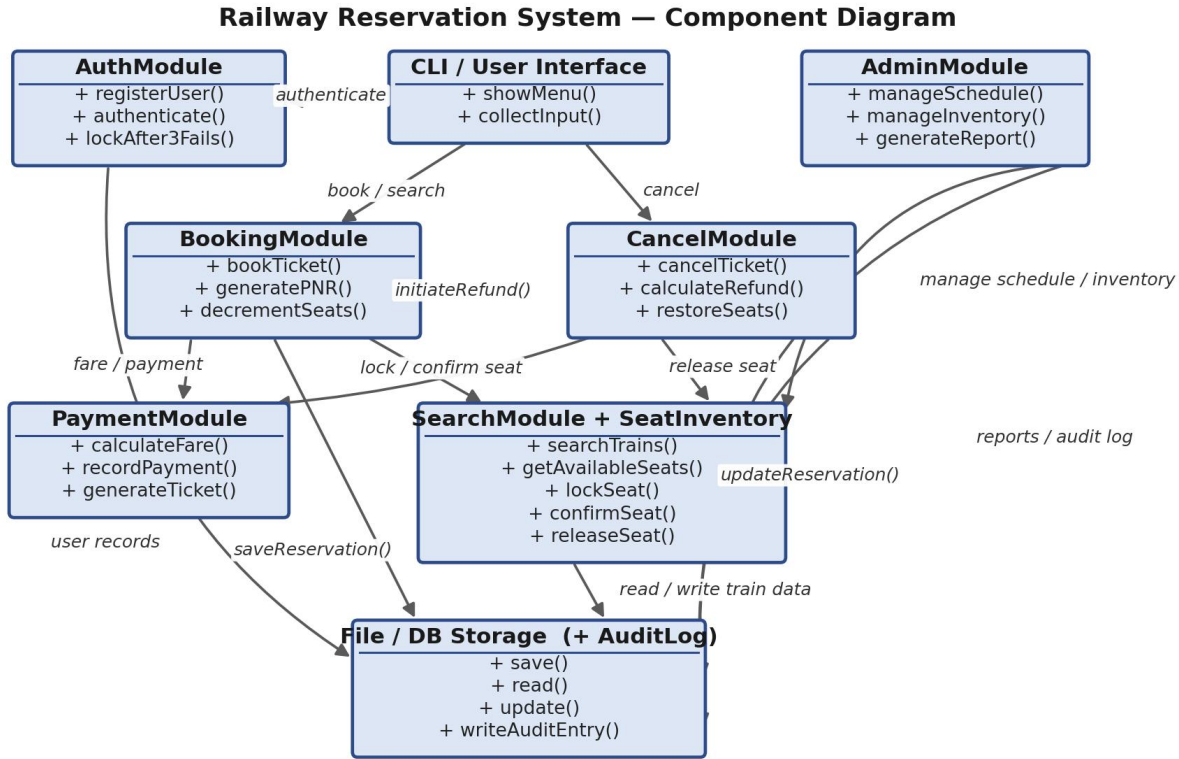


Figure 2: Component diagram — runtime modules mapped to RTM module names

3.4 Component Descriptions

Each module below carries the Design Spec ID used in the RTM (SRS Section 8) and lists the requirements it realises.

- CLI / User Interface — menu-driven console UI with numbered options and input validation (SRS 3.1). Runs a passenger menu and a separate admin menu. Realises RRS-NF-004.
- AuthModule (DS-Auth-01, DS-Auth-02) — passenger registration, credential authentication, and lockout after 3 consecutive failed attempts. Stores only hashed passwords. Realises RRS-F-001, RRS-F-002, RRS-F-003, RRS-SR-001.
- SearchModule + SeatInventory (DS-Search-01) — train search by source, destination and date; per-class availability (Sleeper/AC/General); alternative-train suggestion when full; seat lock/confirm/release. Realises RRS-F-004, RRS-F-005, RRS-F-006, RRS-F-010, RRS-F-011, RRS-F-016, RRS-NF-001, RRS-NF-002.
- BookingModule (DS-Book-01, DS-Book-02) — seat/berth preference capture, booking creation, unique PNR generation, seat-count decrement, and booking-history retrieval. Realises RRS-F-007, RRS-F-008, RRS-F-009, RRS-F-010, RRS-F-011, RRS-F-019.
- PaymentModule (DS-Pay-01) — fare calculation from class, route and passenger count; records simulated payment mode (Cash/Card/UPI); generates the printable ticket. Realises RRS-F-012, RRS-F-013, RRS-F-014.
- CancelModule (DS-Cancel-01) — PNR-based cancellation, seat restoration to inventory, refund calculation per the cancellation-charge policy, and refund-status lookup. Realises RRS-F-015, RRS-F-016, RRS-F-017, RRS-F-018, RRS-SR-005.
- AdminModule (DS-Admin-01) — train schedule CRUD, seat-inventory configuration, fare/quota rules, and occupancy/revenue reporting. Access restricted to authenticated admins. Realises RRS-F-020, RRS-F-021, RRS-F-022, RRS-SR-002.

- File/DB Storage + AuditLog (DS-Sec-01 for audit) — flat-file persistence of user, train, seat, booking, payment and cancellation records via C++ fstream (SRS 3.3), plus a timestamped audit log of every state-changing action. Realises RRS-NF-003, RRS-SR-004.

3.5 Chosen Architecture Pattern and Rationale

ADR-001 — Decision: Adopt a three-tier layered architecture: Presentation layer (CLI), Business Logic layer (Booking Manager, Cancellation Manager, Payment Module), and Data Access layer (Train Database Module, File/DB Storage).

Context: The system is a single-machine C/C++ console application with a small, well-understood feature set and no network deployment requirement.

Alternatives considered: A client-server split was rejected because there is no remote access requirement. Microservices were rejected as disproportionate to the project scale and unsupported by the C/C++ console toolchain. A single-file monolithic program was rejected because it would make the booking and cancellation logic difficult to unit-test.

Consequences: Business logic can be tested independently of the CLI, and the storage backend can later be swapped from flat files to SQLite without touching the upper layers. The trade-off is additional indirection between modules.

3.6 Technology Stack & Data Stores

Implemented in C/C++ (GCC/G++) as a console application running on Windows and Linux (RRS-NF-005). Entity classes shown in Figure 1 are realised as C++ classes/structs. Persistence uses flat-file I/O through the fstream interface (SRS 3.3); no external DBMS or network service is required, per SRS 3.4. Admin reports are exported as plain text.

3.7 Risks & Mitigations

Risk (RRS-NF-002): Concurrent-like booking requests could allocate the same seat twice, breaking the invariant that booked seats never exceed available seats.

Mitigation: SeatInventory exposes lockSeat() / confirmSeat() / releaseSeat(). A seat is locked before payment and only confirmed after payment succeeds; a failed payment releases the lock. Verified by TC-Rel-01.

Risk (RRS-NF-003): Data-file corruption if the application crashes mid-write, losing booking records.

Mitigation: Write-to-temporary-file-then-rename for all persistence operations, so a crash leaves the previous good file intact. Verified by TC-Rel-02.

Risk (RRS-F-003 / RRS-SR-001): Credential compromise through brute-force login or plain-text password storage.

Mitigation: Masked password entry, hashed storage, and account lockout after 3 consecutive failures. Verified by TC-Auth-03 and TC-Sec-01.

3.8 Traceability to Requirements

Requirement IDs below are taken directly from SRS v1.0. This table extends the RTM in SRS Section 8 with the design module that realises each requirement.

Req ID	Requirement (short)	Design module
RRS-F-001/002/003	Registration, login, lockout after 3 failures	AuthModule
RRS-F-004/005/006	Train search, per-class availability, alternatives	SearchModule + SeatInventory
RRS-F-007/008	Book ticket, seat/berth preference	BookingModule
RRS-F-009	Unique PNR generation	BookingModule
RRS-F-010/011	Decrement seats, reject over-booking	BookingModule + SeatInventory
RRS-F-012/013	Fare calculation, payment mode recording	PaymentModule
RRS-F-014	Ticket generation and display	PaymentModule

Req ID	Requirement (short)	Design module
RRS-F-015/016	Cancel by PNR, restore seat inventory	CancelModule + SeatInventory
RRS-F-017/018	Refund calculation and refund status	CancelModule
RRS-F-019	View booking history	BookingModule
RRS-F-020/021/022	Schedule, inventory, occupancy/revenue reports	AdminModule
RRS-NF-001	Search response within 2 seconds	SearchModule (in-memory seat index)
RRS-NF-002	Booked seats never exceed available seats	SeatInventory lock/confirm protocol
RRS-NF-003	Records persist across restart	File/DB Storage (atomic write)
RRS-NF-004	Menu usable without documentation	CLI / User Interface
RRS-NF-005	Builds and runs on Windows and Linux	Standard C++ only, no OS-specific APIs
RRS-SR-001	Password masking and hashed storage	AuthModule
RRS-SR-002	Admin functions restricted to admins	AuthModule + AdminModule
RRS-SR-003	Input validation / overflow prevention	CLI + all modules
RRS-SR-004	Timestamped audit log of actions	AuditLog in File/DB Storage
RRS-SR-005	Cancelled/non-existent PNR cannot be reused	CancelModule

3.9 Security Architecture

Security Objectives (SRS 5.1.1)

- Protect passenger credentials and personal details from unauthorized access or disclosure.
- Ensure booking and cancellation records cannot be tampered with by unauthorized users, preserving data integrity.

Threat Modeling (STRIDE)

- Spoofing → credential authentication before booking, and PNR ownership verification before cancellation (RRS-F-002, RRS-SR-005).
- Tampering → restricted file permissions, atomic writes, and a timestamped audit log of every state-changing action (RRS-SR-004).
- Information Disclosure → masked password entry, hashed password storage, and no passenger contact details written to logs (RRS-SR-001).
- Denial of Service → bounded input reads and validation of all menu choices, dates and seat counts to prevent malformed input or buffer overflow from crashing the application (RRS-SR-003).
- Elevation of Privilege → administrative functions gated behind an authenticated admin role; a passenger session cannot reach the admin menu (RRS-SR-002).

4. Design

4.1 Design Overview

The system is organized into layered modules that separate the user interface, business logic, and data access, improving maintainability and testability.

4.2 UML Sequence Diagrams

Two sequence diagrams are provided for the primary flows of the system: the Ticket Booking flow and the Ticket Cancellation flow. Note that in the booking flow the seat is locked before payment is processed and only confirmed afterwards, which implements the concurrency mitigation described in Section 3.7.

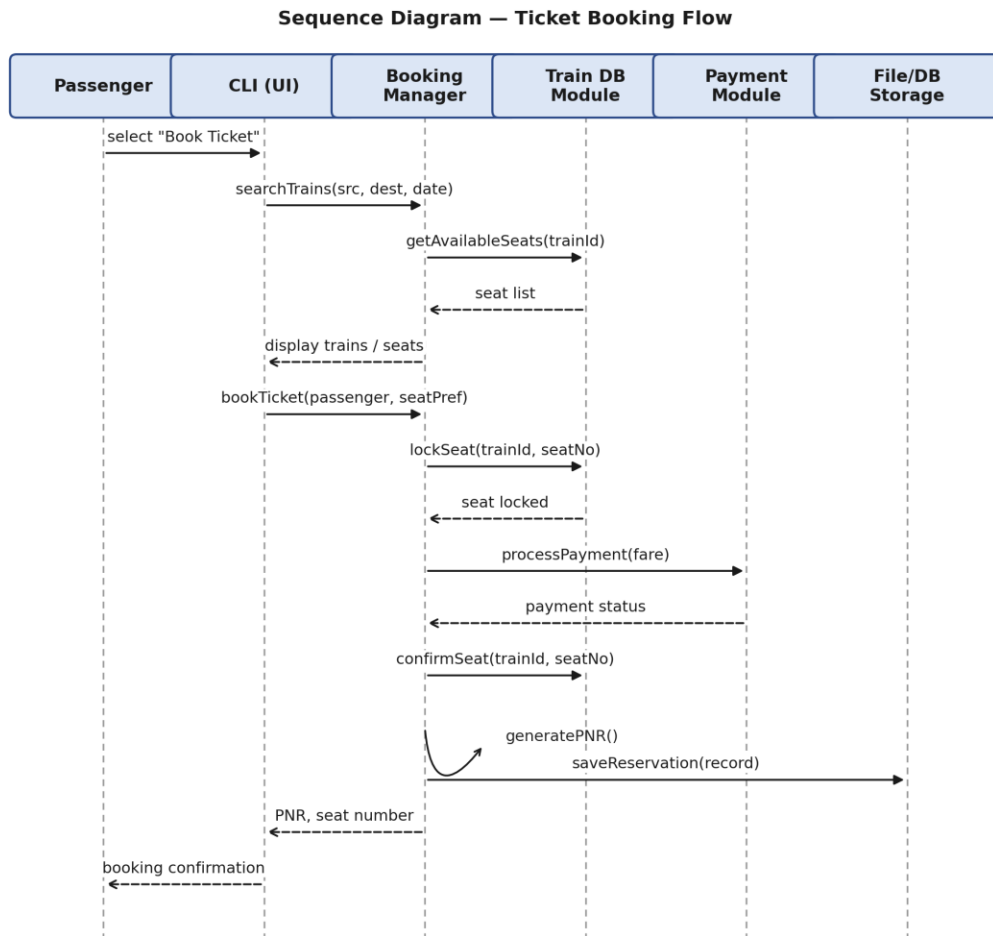


Figure 3: Sequence diagram — Ticket Booking flow

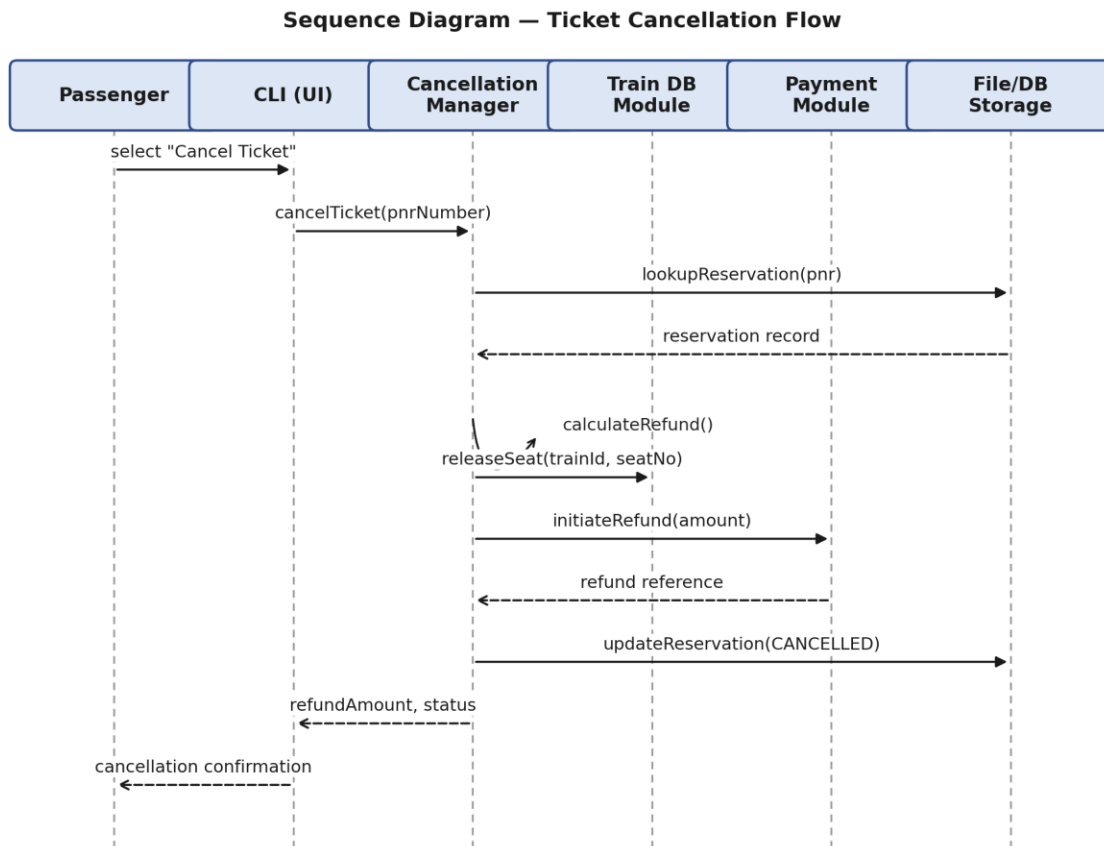


Figure 4: Sequence diagram — Ticket Cancellation flow

4.3 API Design

Interface definitions for four principal modules are given below. Because this is a standalone C/C++ console application rather than a networked service (SRS 3.4), these are expressed as internal module function interfaces rather than HTTP endpoints.

AuthModule (DS-Auth-02)

Function: `authenticate(contact, password)`

Request: {contact, password}

Response: {status, userId, role}

Errors: `ERR_INVALID_CREDENTIALS`, `ERR_ACCOUNT_LOCKED` (after 3 failures, RRS-F-003)

BookingModule (DS-Book-01)

Function: `bookTicket(userId, trainNo, classType, seatCount, berthPref)`

Request: {userId, trainNo, classType, seatCount, berthPref}

Response: {status, pnr, seatNumbers[], fare}

Errors: `ERR_INSUFFICIENT_SEATS` (RRS-F-011), `ERR_INVALID_TRAIN`, `ERR_NOT_AUTHENTICATED`

CancelModule (DS-Cancel-01)

Function: cancelTicket(pnr, userId)

Request: {pnr, userId}

Response: {status, refundAmount, refundStatus}

Errors: ERR_PNR_NOT_FOUND, ERR_ALREADY_CANCELLED (RRS-SR-005),
ERR_NOT_TICKET_OWNER

AdminModule (DS-Admin-01)

Function: updateSchedule(adminId, trainNo, route, timing)

Request: {adminId, trainNo, route, departTime, arriveTime}

Response: {status, updatedTrainNo}

Errors: ERR_NOT_ADMIN (RRS-SR-002), ERR_TRAIN_NOT_FOUND

4.4 Error Handling, Logging & Monitoring

All modules return standardized error codes rather than printing ad-hoc messages, so the CLI can render consistent, user-readable text (RRS-NF-004). Passwords are never logged and passenger contact details are omitted from log output (RRS-SR-001). Every state-changing action — booking, cancellation, and admin schedule or inventory change — is written to the audit log with a timestamp and user ID (RRS-SR-004). Monitored measures include search response time against the 2-second target (RRS-NF-001), booking failure rate, and count of rejected over-booking attempts.

4.5 UX Design

Text-based menu with numbered options for search, booking, cancellation and admin functions (SRS 3.1). Every invalid entry produces a specific message naming the expected input rather than a generic failure. Booking and cancellation both require an explicit confirmation step showing fare or refund amount before the action commits. The target is that a first-time user completes a booking without external documentation, measured by TC-UX-01 (RRS-NF-004).

4.6 Open Issues & Next Steps

Open issues: The cancellation-charge policy table referenced by RRS-F-017 is not yet finalised; refund percentages by time-before-departure must be agreed before TC-Cancel-03 can be executed. The fare master data for RRS-F-012 is likewise pending.

Next steps: Finalise the fare and cancellation-policy tables, then implement modules in the order AuthModule → SearchModule → BookingModule → PaymentModule → CancelModule → AdminModule, so each is testable against its SRS acceptance criteria as it lands.

Future enhancements: Graphical front-end, real payment-gateway settlement, RAC/waitlist promotion, and the client-server socket interface noted as optional in SRS 3.4.

5. Appendices

5.1 Glossary: PNR (Passenger Name Record), RAC (Reservation Against Cancellation), ADR (Architecture Decision Record), TDB (Train Database), CLI (Command Line Interface).

5.2 References: IEEE 42010 (Architecture Description), OWASP Secure Coding Practices, NIST SP 800-160 (Systems Security Engineering), C/C++ language standard references.

5.3 Tools: draw.io (use-case diagrams, SRS Section 7), GCC/G++, GDB, Valgrind, Code::Blocks / VS Code.